MATH FOUNDATIONS

# Calculus for Machine Learning

Derivatives, gradients, and the chain rule — the engine that trains neural networks.

**ANIMATED WALKTHROUGH**

Step through pages to play the animation

STEP 1/6

# The Derivative = Instantaneous Slope

f'(x): how fast f changes as x changes, right now.

**f'(x) = limit of [f(x+h) - f(x)] / h  as h -> 0**

slope of the tangent line at x

**f'(x) > 0**

going up — increase x to grow f

**f'(x) < 0**

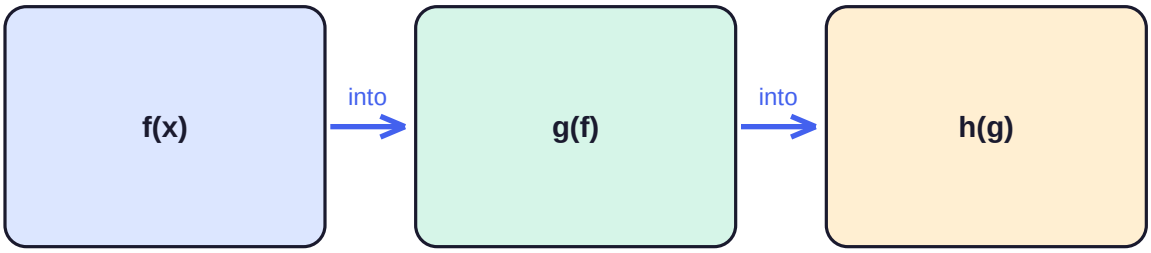
going down — decrease x to grow f

Optimization is just: follow the sign of the derivative until it hits zero.

STEP 2/6

# The Chain Rule

Derivatives of nested functions multiply — this single rule trains deep nets.




**d/dx h(g(f)) = h'(g) * g'(f) * f'(x)**

peel layers outside-in, multiply each local slope

A 100-layer network is just a 100-deep nesting. The chain rule handles arbitrary depth.

STEP 3/6

# Partial Derivatives

With many inputs, vary one at a time and freeze the rest.

**df/dx treats y, z, ... as constants**

neural nets have millions of parameters — one partial each

**df/dx > 0**

nudge x up, f grows

**df/dy < 0**

nudge y up, f shrinks

Each weight in a network gets its own partial derivative: its personal 'which way is down'.

STEP 4/6

# The Gradient & Gradient Descent

Stack all partials into one vector — it points uphill. Walk the other way.

**grad f = [df/dx1, df/dx2, ..., df/dxn]**

points in the direction of steepest INCREASE

**w := w - lr * grad f(w)**

step downhill; lr = learning rate (step size)

**lr too big -> diverge  lr too small -> crawl**

tuning lr is half of practical ML

STEP 5/6

# Loss Landscapes

Training = descending a high-dimensional terrain to minimize loss.

**GLOBAL MINIMUM**

best possible loss — rarely found, rarely needed

**LOCAL MINIMUM**

valley, not the deepest — usually good enough

**SADDLE POINT**

flat in some directions, sloped in others — the real trap

**PLATEAU**

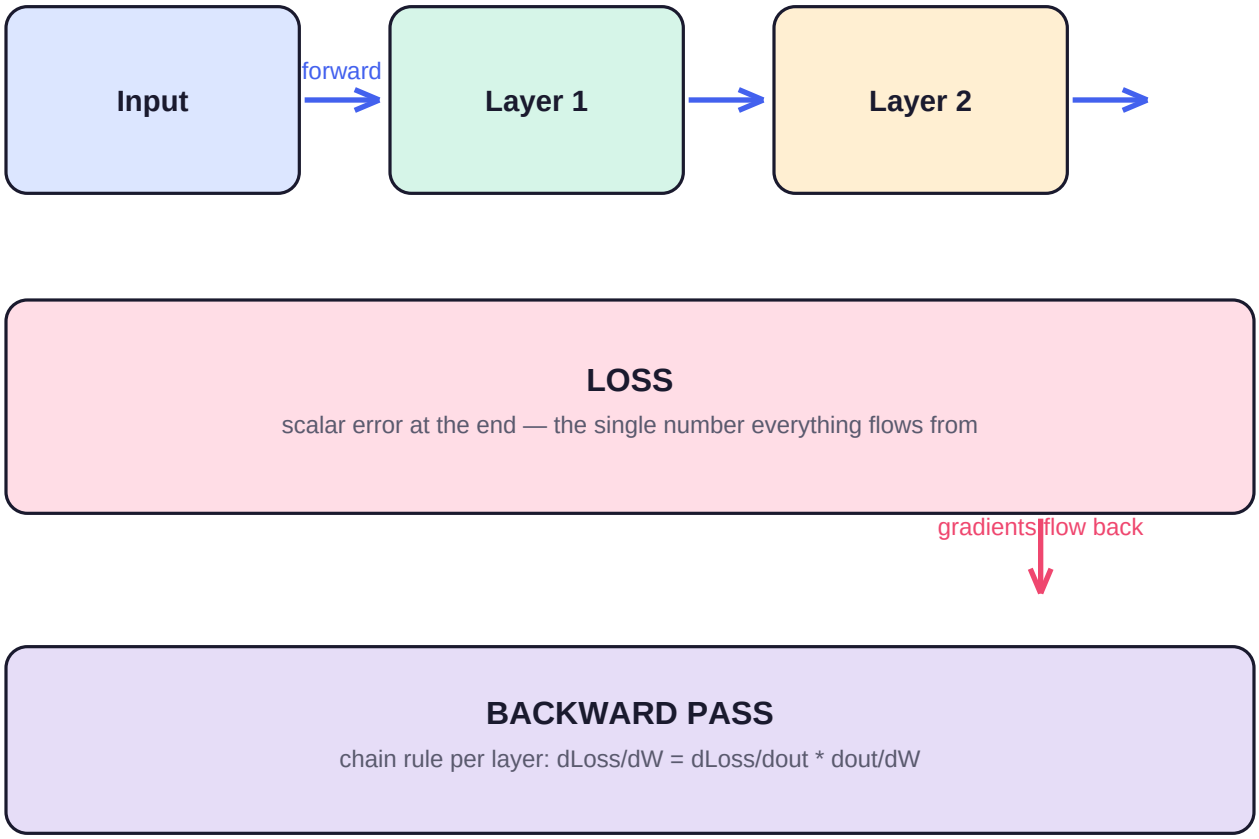
tiny gradients — progress stalls; momentum helps

Deep nets have zillions of 'good enough' minima. Finding one of them is the whole job.

STEP 6/6

# Backprop = Chain Rule at Scale

One forward pass, then sweep gradients backward layer by layer.



STEP ✓/✓

# Key Takeaways

## Calculus for ML — Remember

- The derivative is instantaneous slope; its sign says which way is up.
- The chain rule multiplies local slopes through nested functions — the core of deep learning.
- Partial derivatives isolate one parameter's effect while freezing the rest.
- The gradient points uphill; gradient descent walks downhill with step size lr.
- Loss landscapes hold minima, saddles, and plateaus — local minima are usually good enough.
- Backpropagation is the chain rule applied layer-by-layer, forward once then backward.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev